We haven't updated our specs yet. If we run them now, the message expectation will fail; the code is calling a different API than we've specified.

We've got a bigger problem, though. Our old specs don't stub out the bulk_charge_cards method. We'll end up sending a *real* charge request—something we don't want to do from a unit spec!

The partial double made it easy for our code to perform a costly operation for real, even though the entire point of the test double was to prevent this problem.

We can easily avoid this problem by using a pure or verifying test double instead. To do so, we'll add a new argument to process_subscriptions that lets us inject a billing object other than CashCow:

15-using-test-doubles-effectively/09/subscription_service/lib/recurring_payment.rb

```
class RecurringPayment
  def self.process_subscriptions(subscriptions, bank: CashCow)
    subscriptions.each do |subscription|
      bank.charge_card(subscription.credit_card, subscription.amount)
      # ...send receipt and other stuff...
    end
  end
end
```

Now, instead of stubbing methods on the real CashCow class, our specs can just create a verifying double and pass it in instead:

15-using-test-doubles-effectively/09/subscription_service/spec/recurring_payment_spec.rb

```
RSpec.describe RecurringPayment do
  it 'charges the credit card for each subscription' do
    card_1 = Card.new(:visa, '1234 5678 9012 3456')
    card_2 = Card.new(:mastercard, '9876 5432 1098 7654')

    subscriptions = [
      Subscription.new('John Doe', card_1, 19.99),
      Subscription.new('Jane Doe', card_2, 29.99)
    ]

    bank = class_double(CashCow)
    expect(bank).to receive(:charge_card).with(card_1, 19.99)
    expect(bank).to receive(:charge_card).with(card_2, 29.99)

    RecurringPayment.process_subscriptions(subscriptions, bank: bank)
  end
end
```

This kind of substitution is only feasible if you want an entirely fake object. If you find yourself needing to mix real and fake behavior in the same object, consider splitting it into multiple objects, as we did in the previous section.